

Gradient Descent

Hoa Vu
CS 663

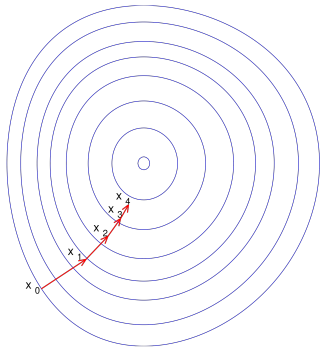
SDSU

April 8, 2026

Maximization and Minimization

Minimize function say $f(x, y) = 2e^{-x^2+1} + y^2$.

Gradient Descent/Ascent



At each step, take a small step in the direction that is steepest descent/ascent.

Gradient Descent

It turns out that the direction of steepest ascent is given by the gradient

$$\nabla f(x, y) = \begin{pmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{pmatrix}.$$

For example, $f(x, y) = 2e^{-x^2+1} + y^2$ has the gradient:

$$\nabla f(x, y) = \begin{pmatrix} -4x e^{-x^2+1} \\ 2y \end{pmatrix}.$$

Gradient Descent

To minimize f , we can use the following update rule:

$$\begin{pmatrix} x \\ y \end{pmatrix} \leftarrow \begin{pmatrix} x \\ y \end{pmatrix} - \alpha \nabla f(x, y),$$

where $\alpha > 0$ is the learning rate.

Gradient Descent

$f(x, y) = 2e^{-x^2+1} + y^2$ has the gradient:

$$\nabla f(x, y) = \begin{pmatrix} -4x e^{-x^2+1} \\ 2y \end{pmatrix}.$$

Suppose we start at $(x, y) = (1, 1)$

$$f(1, 1) = 2e^{-1^2+1} + 1^2 = 3.$$

Gradient Descent

Use a learning rate of $\alpha = 0.1$. Then:

$$\nabla f(1, 1) = \begin{pmatrix} -4(1)e^0 \\ 2(1) \end{pmatrix} = \begin{pmatrix} -4 \\ 2 \end{pmatrix}.$$

The first update is:

$$\begin{pmatrix} x \\ y \end{pmatrix} \leftarrow \begin{pmatrix} 1 \\ 1 \end{pmatrix} - 0.1 \begin{pmatrix} -4 \\ 2 \end{pmatrix} = \begin{pmatrix} 1.4 \\ 0.8 \end{pmatrix}.$$

Then,

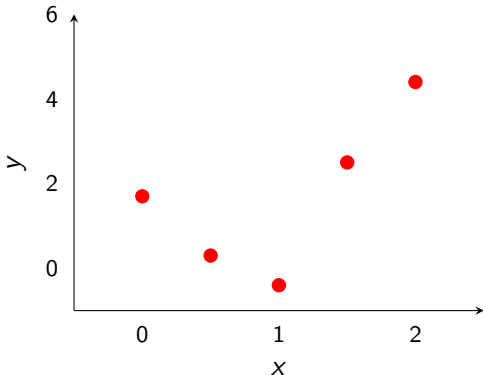
$$f(1.4, 0.8) = 2e^{-0.96} + 0.64 \approx 1.406 < 3 = f(1, 1).$$

Fitting a curve

We have 6 data points (x_i, y_i) :

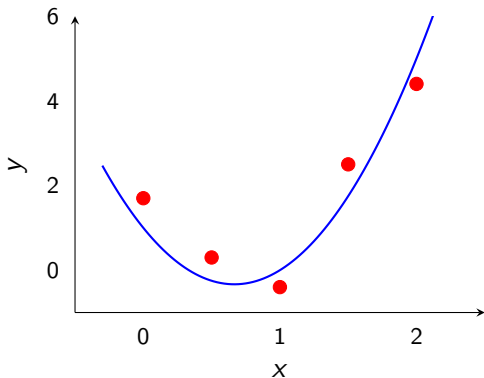
$(0, 1.7), (0.5, 0.3), (1, -0.4), (1.5, 2.5), (2, 4.4)$

We suspect that the data follows a quadratic relationship.



Fitting a curve

Find a quadratic function $y = ax^2 + bx + c$ that best fits the data.



Squared error loss

We can measure how well the curve fits the data using the squared error loss:

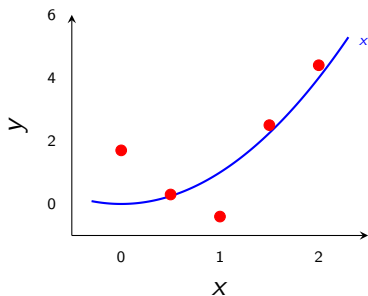
$$L(a, b, c) = \sum_{i=1}^n (y_i - (ax_i^2 + bx_i + c))^2.$$

We want to find a, b, c that minimize $L(a, b, c)$.

$$(y_i - (ax_i^2 + bx_i + c))^2$$

is the squared distance from the point y_i to the curve at x_i .

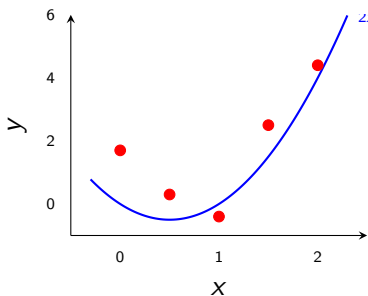
Example: $a = 1$, $b = 0$, $c = 0$



$$f(x) = x^2$$

$(y_i - f(x_i))^2$	
$(1.7 - 0)^2$	$= 2.89$
$(0.3 - 0.25)^2$	$= 0.00$
$(-0.4 - 1)^2$	$= 1.96$
$(2.5 - 2.25)^2$	$= 0.06$
$(4.4 - 4)^2$	$= 0.16$
L	≈ 5.08

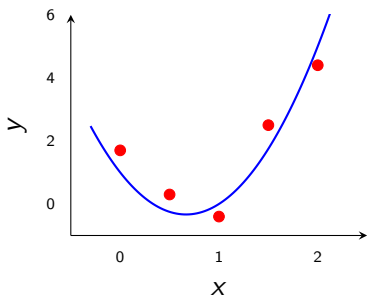
Example: $a = 2$, $b = -2$, $c = 0$



$$f(x) = 2x^2 - 2x$$

$(y_i - f(x_i))^2$	
$(1.7 - 0)^2$	$= 2.89$
$(0.3 - (-0.5))^2$	$= 0.64$
$(-0.4 - 0)^2$	$= 0.16$
$(2.5 - 1.5)^2$	$= 1.00$
$(4.4 - 4)^2$	$= 0.16$
L	≈ 4.85

Example: $a = 3$, $b = -4$, $c = 1$



$$f(x) = 3x^2 - 4x + 1$$

$(y_i - f(x_i))^2$	
$(1.7 - 1)^2$	$= 0.49$
$(0.3 - (-0.25))^2$	$= 0.30$
$(-0.4 - 0)^2$	$= 0.16$
$(2.5 - 1.75)^2$	$= 0.56$
$(4.4 - 5)^2$	$= 0.36$
L	≈ 1.88

Gradient Descent

$$L(a, b, c) = \sum_{i=1}^n (y_i - (ax_i^2 + bx_i + c))^2.$$

The gradient of L is:

$$\nabla L = \begin{pmatrix} \frac{\partial L}{\partial a} \\ \frac{\partial L}{\partial b} \\ \frac{\partial L}{\partial c} \end{pmatrix} = -2 \sum_{i=1}^n (y_i - ax_i^2 - bx_i - c) \begin{pmatrix} x_i^2 \\ x_i \\ 1 \end{pmatrix}.$$

Gradient Descent

Therefore, the update rule is:

$$\begin{pmatrix} a \\ b \\ c \end{pmatrix} \leftarrow \begin{pmatrix} a \\ b \\ c \end{pmatrix} + 2\alpha \sum_{i=1}^n (y_i - ax_i^2 - bx_i - c) \begin{pmatrix} x_i^2 \\ x_i \\ 1 \end{pmatrix},$$

where $\alpha > 0$ is the learning rate.

The Cost of Gradient Descent

Each update requires computing the full gradient:

$$\nabla L = -2 \sum_{i=1}^n (y_i - ax_i^2 - bx_i - c) \begin{pmatrix} x_i^2 \\ x_i \\ 1 \end{pmatrix}.$$

- ▶ Cost per update: $O(n)$.
- ▶ When $n = 10^9$ (e.g. web-scale data), one step is very expensive.
- ▶ **Idea:** replace the full sum with a *single* random sample.

Stochastic Gradient Descent (SGD)

Pick one sample (x_i, y_i) uniformly at random. Use it as a stand-in for the full gradient:

$$\nabla L \approx -2(y_i - ax_i^2 - bx_i - c) \begin{pmatrix} x_i^2 \\ x_i \\ 1 \end{pmatrix}.$$

SGD update rule (one sample per step):

$$\begin{pmatrix} a \\ b \\ c \end{pmatrix} \leftarrow \begin{pmatrix} a \\ b \\ c \end{pmatrix} + 2\alpha (y_i - ax_i^2 - bx_i - c) \begin{pmatrix} x_i^2 \\ x_i \\ 1 \end{pmatrix}.$$

Cost per update: $O(1)$ instead of $O(n)$.

Why Does It Work?

The single-sample gradient is an **unbiased estimator** of the full gradient:

$$\mathbb{E}_i \left[-2(y_i - ax_i^2 - bx_i - c) \begin{pmatrix} x_i^2 \\ x_i \\ 1 \end{pmatrix} \right] = \nabla L.$$

- ▶ Each step is *noisy*, but on average points in the right direction.
- ▶ Many cheap noisy steps $\gg$ one expensive exact step.

SGD: Worked Example

Data: $(0, 1.7)$, $(0.5, 0.3)$, $(1, -0.4)$, $(1.5, 2.5)$, $(2, 4.4)$.

Start at $(a, b, c) = (1, 0, 0)$, $\alpha = 0.05$.

Step 1: pick sample $(x_1, y_1) = (1, -0.4)$.

Residual: $y_1 - (1 \cdot 1^2 + 0 \cdot 1 + 0) = -0.4 - 1 = -1.4$.

$$\begin{pmatrix} a \\ b \\ c \end{pmatrix} \leftarrow \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} + 2(0.05)(-1.4) \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 0.86 \\ -0.14 \\ -0.14 \end{pmatrix}.$$

Step 2: pick sample $(x_2, y_2) = (2, 4.4)$, and repeat.

Mini-Batch SGD

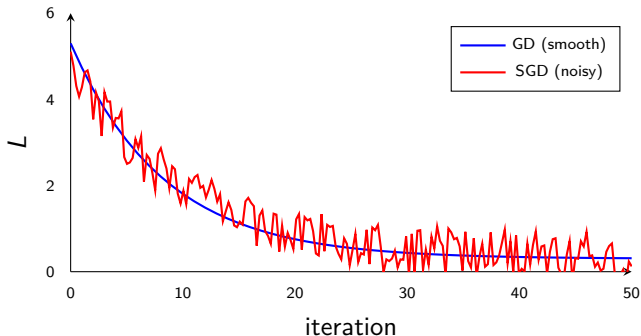
Compromise: use a random batch $\mathcal{B} \subset \{1, \dots, n\}$ of size B .

$$\begin{pmatrix} a \\ b \\ c \end{pmatrix} \leftarrow \begin{pmatrix} a \\ b \\ c \end{pmatrix} + \frac{2\alpha}{B} \sum_{i \in \mathcal{B}} (y_i - ax_i^2 - bx_i - c) \begin{pmatrix} x_i^2 \\ x_i \\ 1 \end{pmatrix}.$$

	GD	Mini-batch	SGD
Batch size	n	B	1
Cost/step	$O(n)$	$O(B)$	$O(1)$
Noise	none	low	high

Typical values: $B \in \{32, 64, 128, 256\}$.

GD vs SGD: Convergence



- ▶ GD converges smoothly but each step costs $O(n)$.
- ▶ SGD oscillates but reaches a good solution much faster.

Learning Rate Schedule

SGD with a **fixed** α oscillates forever near the minimum.

Fix: decay α over time so steps shrink.

Robbins–Monro convergence conditions:

$$\underbrace{\sum_{t=1}^{\infty} \alpha_t = \infty}_{\text{steps large enough to reach min}} \qquad \underbrace{\sum_{t=1}^{\infty} \alpha_t^2 < \infty}_{\text{noise dies out}}$$

Three common schedules: **step decay**, **$1/t$ decay**, **cosine annealing**.

Schedule 1: Step Decay

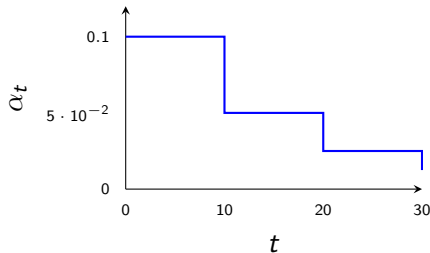
$$\alpha_t = \alpha_0 \cdot \gamma^{\lfloor t/k \rfloor}, \quad 0 < \gamma < 1.$$

- ▶ Every k steps, multiply α by γ .

- ▶ **Example:** $\alpha_0 = 0.1$,
 $\gamma = 0.5$, $k = 10$.

- ▶ Steps 0–9: $\alpha = 0.1$
- ▶ Steps 10–19: $\alpha = 0.05$
- ▶ Steps 20–29: $\alpha = 0.025$

- ▶ Simple and widely used in deep learning.
- ▶ **Does not** satisfy Robbins–Monro ($\alpha_t \rightarrow c > 0$).



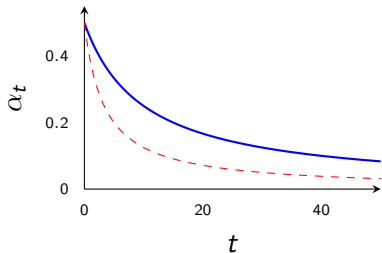
Schedule 2: $1/t$ Decay

$$\alpha_t = \frac{\alpha_0}{1 + \beta t}.$$

- ▶ Decays continuously and smoothly.
- ▶ β controls how fast α shrinks.
- ▶ **Satisfies** Robbins–Monro:

$$\sum \frac{1}{t} = \infty, \quad \sum \frac{1}{t^2} < \infty.$$

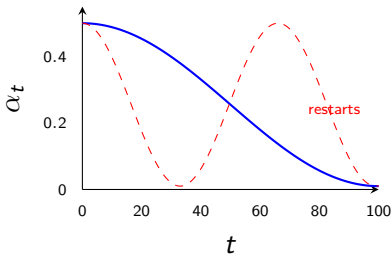
- ▶ Provably converges to a local minimum.
- ▶ Can be slow early on if β is too large.



Schedule 3: Cosine Annealing

$$\alpha_t = \alpha_{\min} + \frac{1}{2}(\alpha_0 - \alpha_{\min}) \left(1 + \cos \frac{t\pi}{T} \right).$$

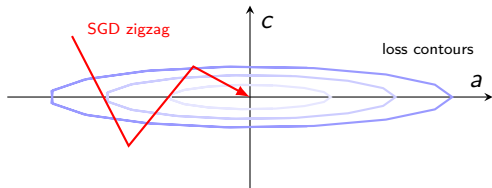
- ▶ Smoothly decays from α_0 to $\alpha_{\min}$ over T steps.
- ▶ Often restarted periodically (**warm restarts**).
- ▶ Popular in neural network training.
- ▶ Does **not** satisfy Robbins–Monro, but works well in practice.



Motivation: One Rate Fits None

A single global α treats all parameters equally.

- ▶ Some parameters have **large, frequent** gradients $\Rightarrow$ need small α .
- ▶ Others have **small, rare** gradients $\Rightarrow$ need large α .



Adaptive methods: give each parameter its own effective learning rate.

AdaGrad

Idea: scale α down for parameters with large historical gradients.

Let $g_t = \nabla L_t$ be the gradient at step t . Accumulate squared gradients:

$$G_t = G_{t-1} + g_t^2.$$

Update each parameter θ component-wise:

$$\theta \leftarrow \theta - \frac{\alpha}{\sqrt{G_t} + \epsilon} g_t.$$

- ▶ $\epsilon \approx 10^{-8}$: small constant for numerical stability.
- ▶ Frequent parameters $\Rightarrow$ large $G_t \Rightarrow$ smaller effective α .
- ▶ **Problem:** G_t only grows $\Rightarrow$ learning rate shrinks to zero, stopping learning.

AdaGrad: Numerical Example

Parameters (a, b, c) , $\alpha = 0.1$. After several steps the accumulators are:

$$G_0 = \begin{pmatrix} 16 \\ 1 \\ 0.25 \end{pmatrix} \quad (a \text{ has seen large gradients; } c \text{ has seen small ones}).$$

One more step with gradient $g = (3, 2, 1)^\top$:

	g	$G_0 + g^2$	$\sqrt{G_1}$	eff. $\alpha = 0.1/\sqrt{G_1}$
a	3	$16 + 9 = 25$	5.00	0.020
b	2	$1 + 4 = 5$	2.24	0.045
c	1	$0.25 + 1 = 1.25$	1.12	0.089

RMSProp

Fix AdaGrad: use an *exponential moving average* of squared gradients so old gradients are forgotten.

$$G_t = \rho G_{t-1} + (1 - \rho) g_t^2, \quad \rho \approx 0.9.$$

$$\theta \leftarrow \theta - \frac{\alpha}{\sqrt{G_t} + \epsilon} g_t.$$

- ▶ G_t now tracks a **recent** average of g^2 , not the full history.
- ▶ Learning rate adapts to the current landscape.
- ▶ Typical values: $\alpha = 0.001$, $\rho = 0.9$, $\epsilon = 10^{-8}$.